UNIVERSITY OF MALTA

FACULTY OF INFORMATION & COMMUNICATION TECHNOLOGY

DEPARTMENT OF COMPUTER INFORMATION SYSTEMS

CIS3111 Cloud Computing: 2023/24 Assignment on Secure Cloud Native Computing

Container Orchestration for Cloud-Native Applications.

Instructions:

1. This is an **individual assignment**.
2. You may be required to demonstrate and present your work to an exam board.
3. A soft-copy of the report along with all digital artefacts must be uploaded to the VLE upload area by the deadline below. All files must be archived into a single .zip file. It is the student's responsibility to ensure that the uploaded zip file and all contents are valid.
4. Reports that are difficult to follow due to low quality in the writing-style/organisation/presentation will be penalised.
5. Assignment queries are to be made strictly on the 'Container Orchestration' assignment's forum on VLE.
6. This assignment is marked out of 100 and comprises 30% of the final CIS3111 assessment.
7. You are to allocate 20 hours for this assignment.
8. This assignment comprises three tasks. Ideally, attempt the first task in the period between when the first and second sessions on container orchestration.
9. The report must be organized according to the task sequence and for each, clearly explaining/discussing the solution and referring to the code/script/configuration/output snippets as needed.
10. The firm submission deadline is **31st May 2024 at NOON**.

An online University course enrollment service, *Enrollments*, is currently hosted on a stand-alone server as shown in Figure 1. It consists of an application server and a PostgreSQL database, each hosted in its own docker container to simplify deployment.

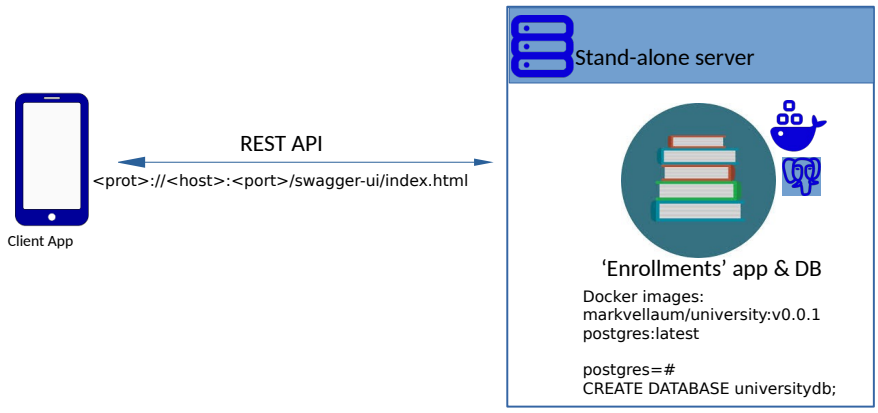

Figure 1: The current *Enrollments* online service hosted on a stand-alone server.

Aiming for high availability, the developers plan to host the *Enrollments* service on a managed Kubernetes platform. The cloud setup, as shown in Figure 2, provides for service discovery, auto-replication, and centralized identity & access management, among other cloud-native architectural benefits for a high-availability and secure deployment.

The development team is currently only in the planning phase of the migration. It is required to replicate aspects of both deployments locally to understand how to proceed.

1. **Task 1: Current deployment** (Total - 30 marks)

   By making use of the official docker documentation[1], and any other sources as deemed relevant:

   (a) Deploy the *Enrollments* application (`markvellaum/university:v0.0.1`) and the PostgreSQL database

[1] https://docs.docker.com/get-started, https://docs.docker.com/engine/reference/run/

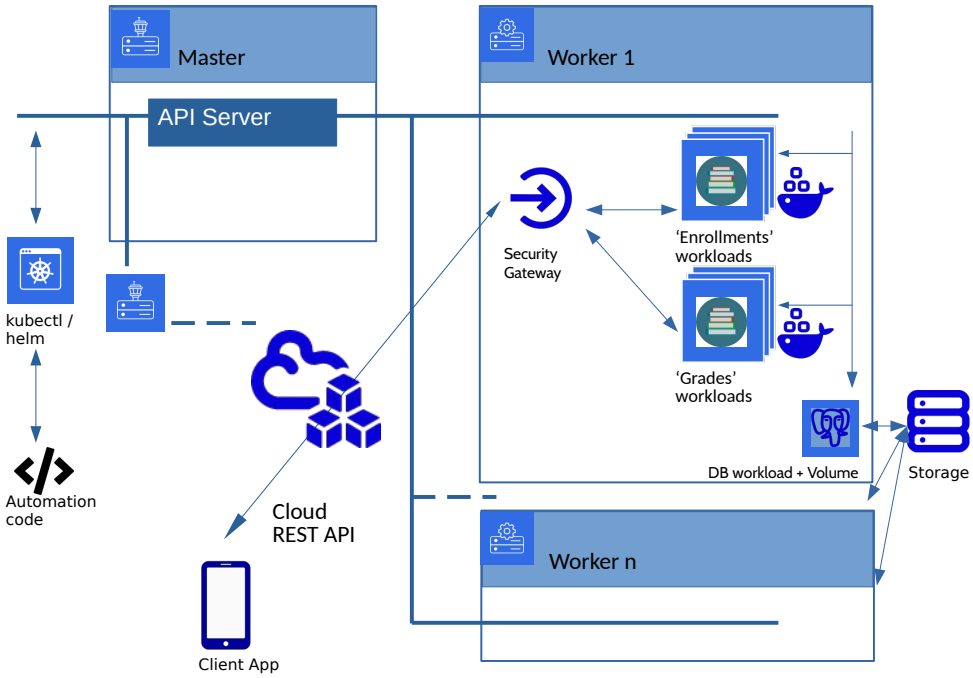

Figure 2: The planned *Enrollments* cloud service deployed on Kubernetes.

(`postgres:latest`) inside two separate docker containers, replicating the deployment in Figure 1. The *Enrollments* application requires the following environment variables: `PGHOST`, `PGPORT`, `PGDB`, `PGUSER`, `PGPASS`, for PostgreSQL's host/port/database/user/password, respectively. The PostgreSQL database, by default, listens on port `5432` and accepts the default user `postgres` and password `pass` on installation. Automate the entire setup using any scripting/programming language of your choice.

[20 marks]

(b) *Enrollments* exposes an OpenAPI documentation of its REST API on the `<prot>://<host>:<port>/swagger-ui/index.html` URI. Import to postman[2] and test the endpoints comprehensively .

[10 marks]

[2]https://www.postman.com/downloads/

Digital artefacts: Code for deployment automation; Test (raw) request/response dumps from postman; Exported PostgreSQL[3] table (post-testing).

2. **Task 2: Kubernetes deployment** (Total - 50 marks)

   By making use of lecture notes, the official documentation for Kubernetes[4] (K8s), minkube[5] and Helm[6], and any other sources as deemed relevant: Migrate the deployment from Figure 1 to minikube, to start preparing for, and test, the target deployment shown in Figure 2. Your solution has to take care of the following:

   (a) Identify and list, along with full explanation, all the necessary K8s ConfigMaps, Secrets, Persistent Volumes, and Persistent Volume Claims API objects.

   [10 marks]

   (b) Perform a Helm installation of PostgreSQL (`riftbit/postgresql`[7]), configured to use the previously identified API objects, to set up a single read/write instance of the database. The database is to be reachable from other workloads within the cluster only.

   [10 marks]

   (c) Configure the deployment of workloads for `markvellaum/university:v0.0.1` - the original variant of *Enrollments* intended for use by the university's admissions department, and `markvellaum/universitygrades:v0.0.1`, a student-portal into *Enrollments*, called *Grades*, intended for students to monitor their current enrollments and the grades obtained for the enrolled courses. The expected traffic ratio for the respective two app variants stands at 1:10. Both are to be reachable by clients external to the cluster.

   [10 marks]

[3] https://www.postgresql.org/docs/current/sql-commands.html, https://www.postgresql.org/docs/current/sql-copy.html

[4] https://kubernetes.io/docs/home/

[5] https://minikube.sigs.k8s.io/docs/start/

[6] https://helm.sh/docs/

[7] https://artifacthub.io/packages/helm/riftbit/postgresql

(d) Automate the entire setup using any scripting/programming language of your choice.

[10 marks]

(e) Test the entire deployment, verifying all cloud-native components are deployed as expected, along with auto replication and self-healing. Use any scripting/programming language of your choice.

[10 marks]

Digital artefacts: All configuration files (including Helm `values.yaml`); Code for deployment automation; Test code/scripts and their outputs.

3. **Task 3: Secured deployment** (Total - 20 marks)

Describe a plan to secure communication within your cluster once hosted at a cloud provider using a security API gateway (see Figure 2). In your answer, you need to:

(a) Identify, and fully explain, the gateway functionality needed to secure the cluster from Internet traffic sniffing, as well as web service spoofing by impostor back-ends.

[10 marks]

(b) Identify, and fully explain, the gateway functionality needed to: i) Restrict access to *Enrollments* (to manage courses/students/enrollments) endpoints only to authorized personnel; and to ii) Restrict access to *Grades* (to view enrolled courses/grades for a specific student) in a way so that students can only view their own grades.

[10 marks]

Digital artefacts: None.

---

end of assignment